

WEEK-01:
Object-Oriented Programming
using Java

Different Programming Paradigms

- Functional/procedural programming:
 - program is a list of instructions to the computer
- Object-oriented programming
 - program is composed of a collection *objects* *that communicate with each other*

Object-Oriented Programming

- In POP. Programs are organized around code
(code acting on data)
- In OOP, code is organized around the data.
(data controlling access to code)
- In OOP, we define the **data and the routines** that are permitted to act on the data.
- In Java, **a class** encapsulates the data and the routines acting on the data.

Main Concepts

- Object
- Class
- Inheritance
- Encapsulation
- Polymorphism
- Method Overloading and Overriding
- this and super keywords
- Constructors
- Abstraction

Objects

- identity – unique identification of an object
- attributes – data/state
- services – methods/operations
 - supported by the object
 - within objects responsibility to provide these services to other clients

Class

- “type”
- object is an **instance** of class
- class groups similar objects
 - same (structure of) attributes
 - same services
- object holds values of its class’s attributes

The **Key traits** of Object Oriented Programming

- *Encapsulation*

- *Polymorphism*

- *Inheritance*

Encapsulation

- ***Encapsulation*** - is the mechanism
 - that **binds together code and the data it manipulates**,
 - and keeps both safe from outside interference and misuse
- ***Encapsulation***
 - Self-contained **black box**
 - **Object** supports encapsulation
- Encapsulation: Java
 - Basic Unit of Encapsulation: **Class**
 - Object: An instance of Class
 - **Class Members**: Instance variables **and** methods

Polymorphism

- ***Polymorphism*** - is a feature that
 - allows **one interface to be used for a general class of actions.**
 - The specific action is determined by the exact nature of the situation.
- ***Example”***
 - **get** method – to retrieve next item from the list
 - List: FIFO, FILO, Based on some priority
- In general
 - One interface, multiple methods
 - A generic interface to a group of related activities
- Helps to reduce complexity
 - Compiler should select specific action as it applies to each situation

Inheritance

- ***Inheritance*** - the process by which one object can acquire the properties of another object
- Supports Hierarchical(top-down) classification
 - Example: Food ← Fruit ← Apple ← Red Delicious Apple
- **Using Inheritance**
 - An object need only define its unique qualities
 - It can inherit its general attributes from its parent

Why Java ?

- Portable
- Easy to learn
- [Designed to be used on the Internet]

Java Buzzwords

- Simple
 - easy to learn, inherits c/c++ syntax, left out difficult constructs like pointers, memory management.
- Pure Object oriented Language-
 - classes and objects
- Java is case sensitive language-
 - variable val is different from Val, VAL, VaL
- Robust-
 - Ability to create robust, reliable programs
 - Exception handling
 - Strictly typed language
(checks for errors at both compile and run time)
 - Easy memory management
(automatic deallocation-garbage collection)

Java Buzzwords

- **Multi threaded**
 - write programs that do many things simultaneously
- **Architecture neutral**
 - compiled java programs will run on variety of processors using various OSs (The goal- “ Write once, run any where, any time, forever”)
- **Both compiled and interpreted**
 - enables creation of cross-platform programs

Java Buzzwords

- **Dynamic**

- Resolves type information at run time

- **Distributed**

- Designed for distributed environment of internet because it handles TCP/IP protocol, RMI

- **Security**

- Execution is confined to JVM

Java and Internet

- Java can be used to create two types of programs: *applications* and *applets*
- An *application* is a program
 - that runs on your computer, under the operating system of that computer
- An *applet* is a tiny Java program
 - designed to be transmitted over the Internet
 - automatically downloaded across the network, just like an image, sound file, or video clip
 - executed by a Java-compatible Web browser

Applets

- Applets
 - Allow some functionality to be moved from server to client like
 - display data provided by the server
 - handle user inputs
 - provide simple functions (loan calculator)
that execute locally rather than on server
- Challenges: Introduces **security** and **portability** issues

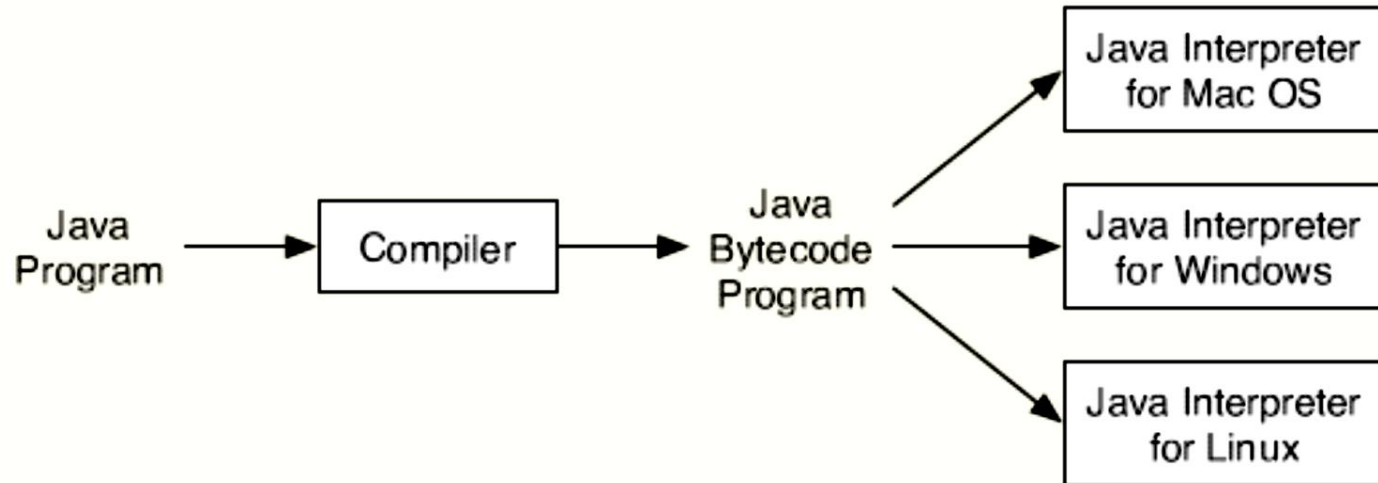
SECURITY

- Automatic downloading of applets might contain virus
 - malicious code gains unauthorized access to system resources, passwords, credit card numbers etc.
- But, when a Java-compatible Web browser used,
 - One can safely download Java applets without fear of viral infection or malicious intent
- **Java achieves this protection**
 - by confining a Java program to the Java execution environment and
 - not allowing access to other parts of the computer environment

Portability

- Portability is the major aspect of Internet
 - because many different types of computers and OSs are connected to it.
- In case of applets,
 - must be able to download and must be executed by wide variety of computers and OSs.
- Because Java is architecture neutral,
 - Java programs are portable.
 - They can be run on any platform without being recompiled.

Java's solution:

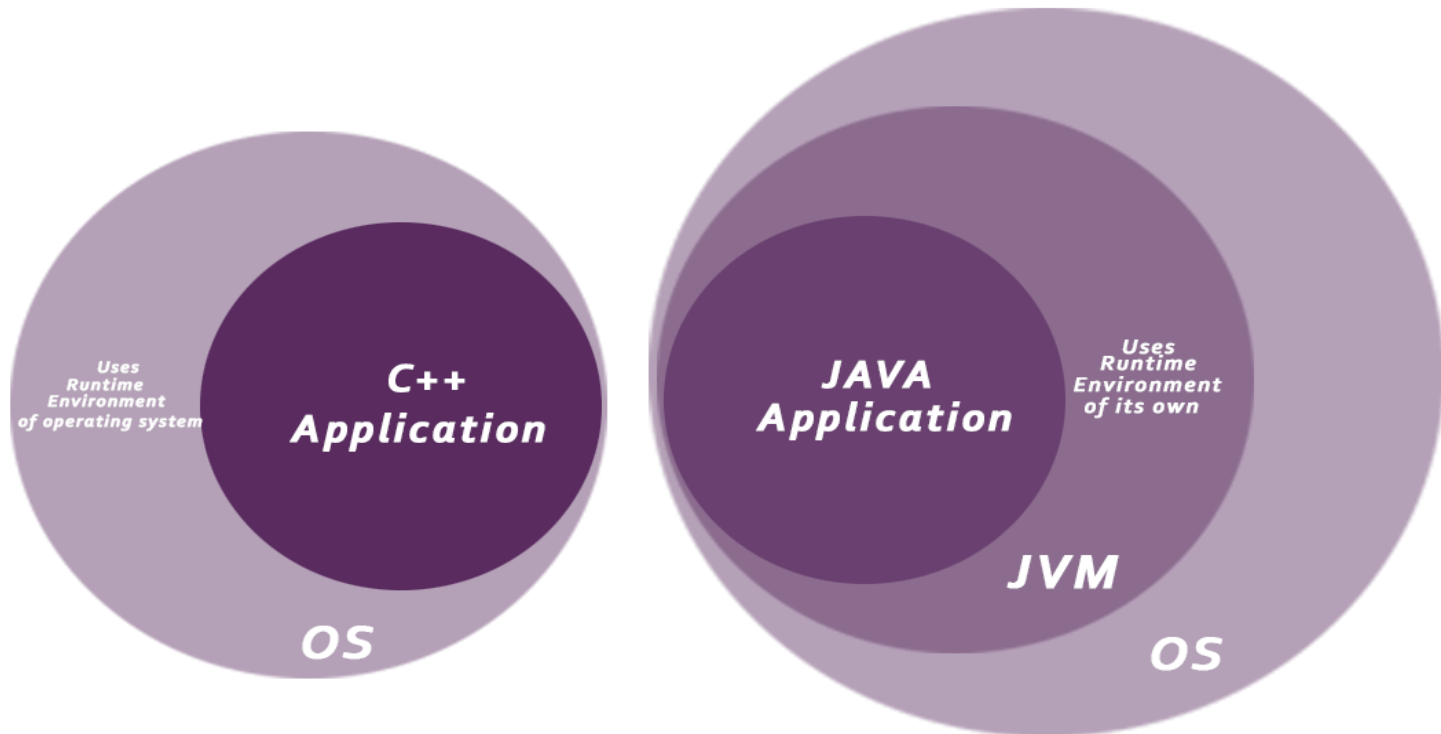


- Execution of **bytecode** by **JVM**
 - is the easiest way to create portable programs by implementing java interpreter for each platform.

Java bytecode

- The output of a Java compiler(javac) is
 - not executable code ; rather, it is *bytecode*
- *Bytecode* is a highly optimized set of instructions
 - designed to be executed by the Java run-time system, *Java Virtual Machine (JVM)*

Java Virtual Machine(JVM)



Java's solution: A Picture is Worth...

Java Program

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java

Compiler

The output of the compiler is .class file

Interpreter

Interpreter

Interpreter



Win32



Solaris



MacOS

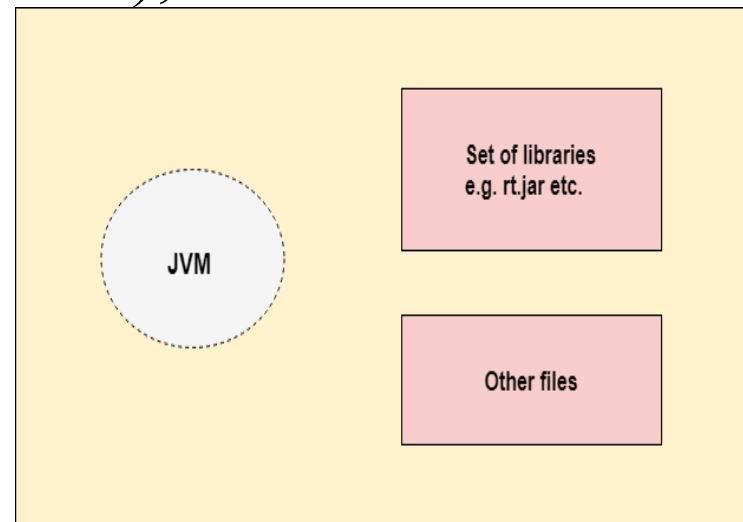
The Interpreter's are sometimes referred to as **Java Virtual Machines**

Just –in-time Compiler(JIT)

- When JIT is part of JVM
 - Selected portions of bytecode are compiled into executable code at run time
 - On a piece-by-piece, demand basis
 - It does not compile an entire java program
 - Furthermore, not all sequences of bytecode are compiled
 - only those that will benefit from compilation

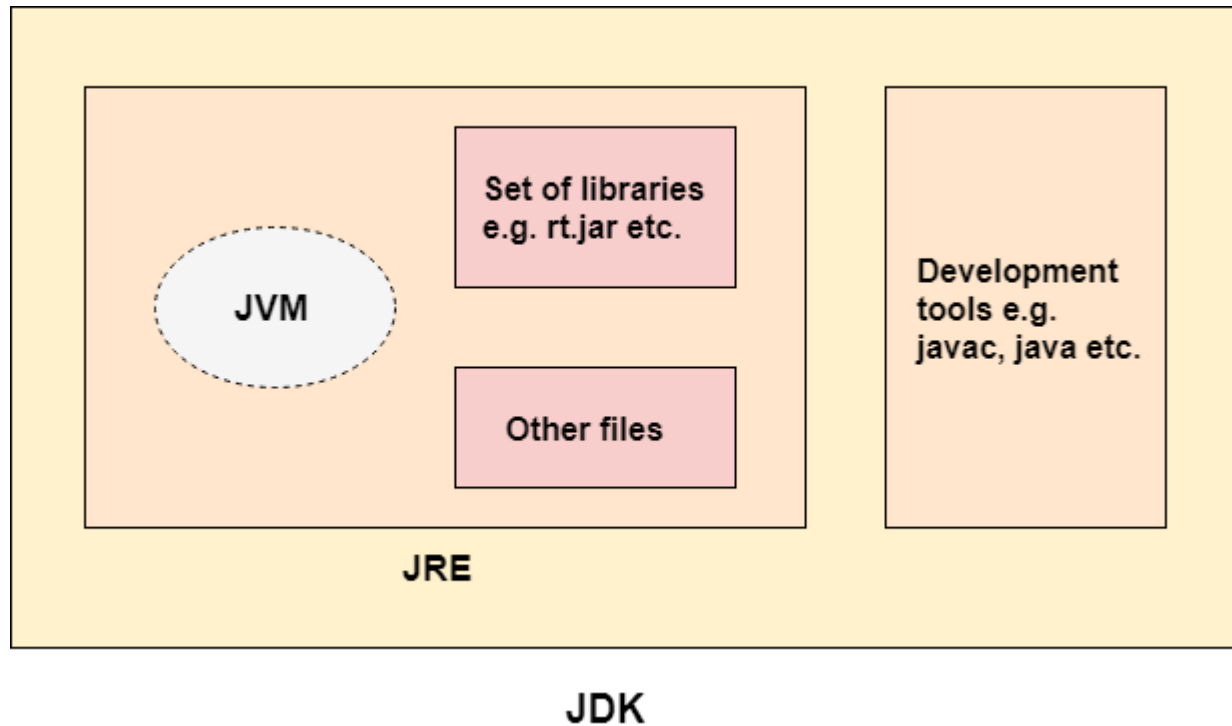
Java Runtime Environment (JRE)

- a set of programming tools for developing Java applications:
 - Java Virtual Machine (JVM),
 - core classes (libraries),
 - supporting files.

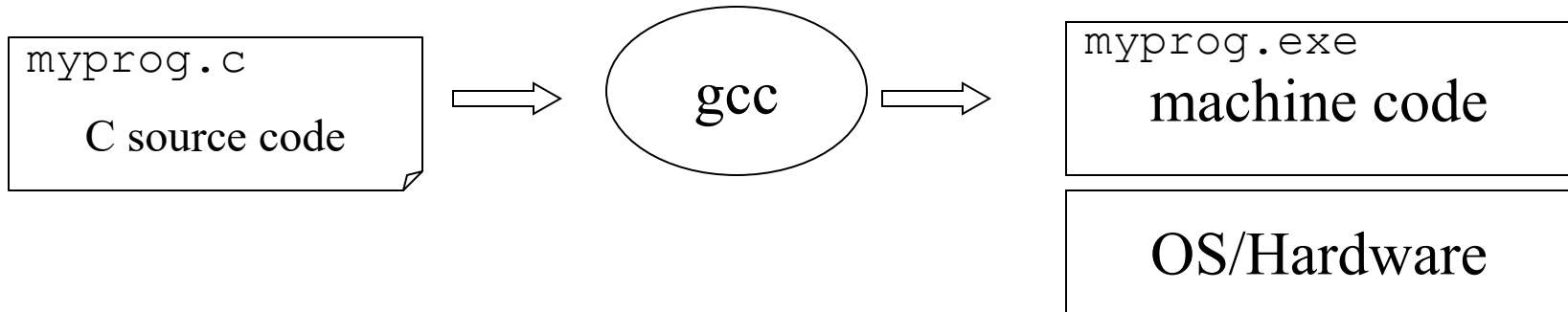


JRE

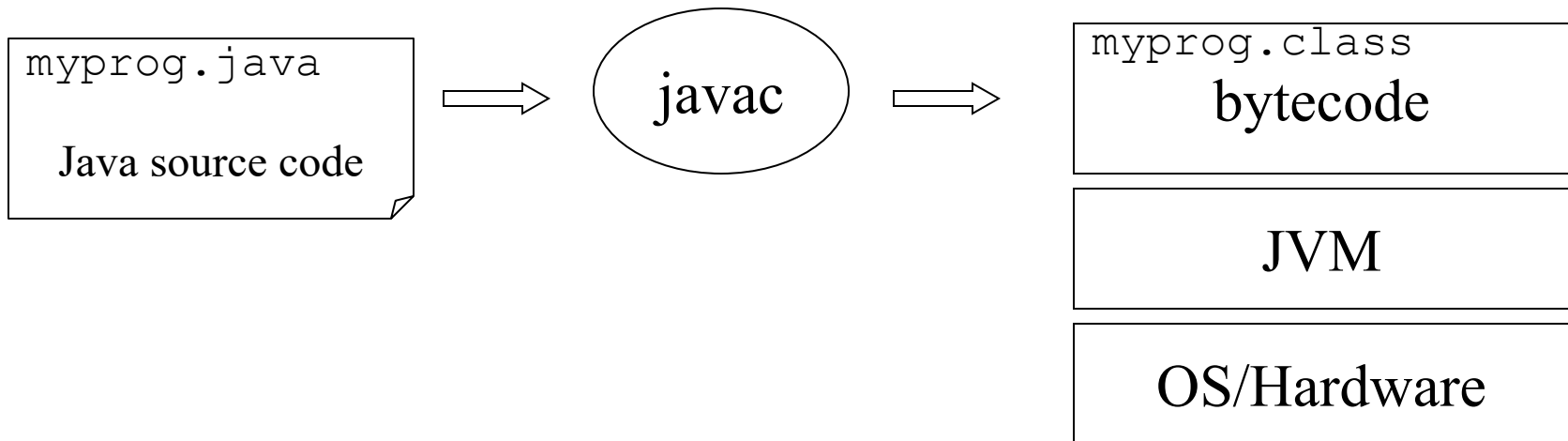
Java Development Kit(JDK)



PLATFORM DEPENDENT



Platform Independent



Primitive types

- int 4 bytes
- short 2 bytes
- long 8 bytes
- byte 1 byte
- float 4 bytes
- double 8 bytes
- char Unicode encoding (2 bytes)
- boolean {true,false}

*Behaviors is
exactly as in
C++*

Note:
*Primitive type
always begin
with lower-case*

Java Tokens

Smallest individual unit in source code.

- Keywords
- Identifiers
- Literals
- Operators
- Separators

Java Data types

- Data types in Java are classified into two types:
 - Primitive—which include Integer, Character, Boolean, and Floating Point.
 - Non-primitive—which include Classes, Interfaces, and Arrays.
- **Primitive Data Types**
 - **1. Integer**
 - Integer types can hold whole numbers such as 123 and –96. The size of the values that can be stored depends on the integer type that we choose.
 - **2. Floating Point**
 - Floating point data types are used to represent numbers with a fractional part. Single precision floating point numbers occupy 4 bytes and Double precision floating point numbers occupy 8 bytes. There are two subtypes:
 - **3. Character**
 - It stores character constants in the memory. It assumes a size of 2 bytes, but basically it can hold only a single character because char stores unicode character sets. It has a minimum value of ‘u0000’ (or 0) and a maximum value of ‘uffff’ (or 65,535, inclusive).
 - **4. Boolean**
 - Boolean data types are used to store values with two states: true or false.

Java's "Building Blocks"

- Java programs relies on software components called objects
 - An object contains both data and behavior
 - An object is defined by a class
- A program is made up of one or more classes
 - A class contains one or more methods
 - A method contains program statements
- Statements are
 - Variable declarations: primitive data types and classes
 - Operations: arithmetic, logical, bit-level, class access
 - Control structures: selection, looping, etc.
 - Object messages: i.e., calls to methods
- A Java application always executes the main method

Primitive types - cont.

- **Constants**

37 integer

37.2 float

42F float

0754 integer (octal)

0xfe integer (hexadecimal)

Java Versus C++

- Java is a true object oriented language while C++ is basically C with an object oriented extension. Listed below are some major C++ features that were intentionally omitted from Java, or significantly modified:
- Java does not support operator overloading.
- Java does not have template classes.
- Java does not support multiple inheritance of classes.
- Java does not support global variables.
- Java does not use pointers.
- Java has replaced the destructor function with the finalize() function.
- There are no header files in Java.

Wrappers

Java provides Objects which wrap primitive types and supply methods.

Example:

```
Integer n = new Integer("4");  
int m = n.intValue();
```

[Read more about Integer in JDK Documentation](#)

Hello World

Hello.java

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World !!!");  
    }  
}
```

C:\javac Hello.java

(*compilation creates Hello.class*)

C:\java Hello

(*Execution on the local JVM*)

More sophisticated

```
class Kyle {  
    private boolean kennyIsActive_  
    public Kyle() { kennyIsActive_ = true; }  
    public Kyle(Kyle aKyle) {  
        kennyIsActive_ = aKyle.kennyIsActive_  
    }  
    public String theyKilledKenny() {  
        if (kennyIsActive_) {  
            kennyIsActive_ = false;  
            return "You are Welcome !!!";  
        } else {  
            return "?";  
        }  
    }  
    public static void main(String[] args) {  
        Kyle k = new Kyle();  
        String s = k.theyKilledKenny();  
        System.out.println("Kyle: " + s);  
    }  
}
```

Default
C'tor

Copy
C'tor

Results

```
javac Kyle.java      ( to compile )
```

```
java Kyle            ( to execute )
```

```
Kyle: You are Welcome !!!
```

Arrays

- Array is an object
- Array size is fixed

```
Animal[] arr; // nothing yet ...
```

```
arr = new Animal[4]; // only array of pointers
```

```
for(int i=0 ; i < arr.length ; i++) {  
    arr[i] = new Animal();  
}
```

```
// now we have a complete array
```

Arrays - Multidimensional

- In C++

```
Animal arr[2][2]
```

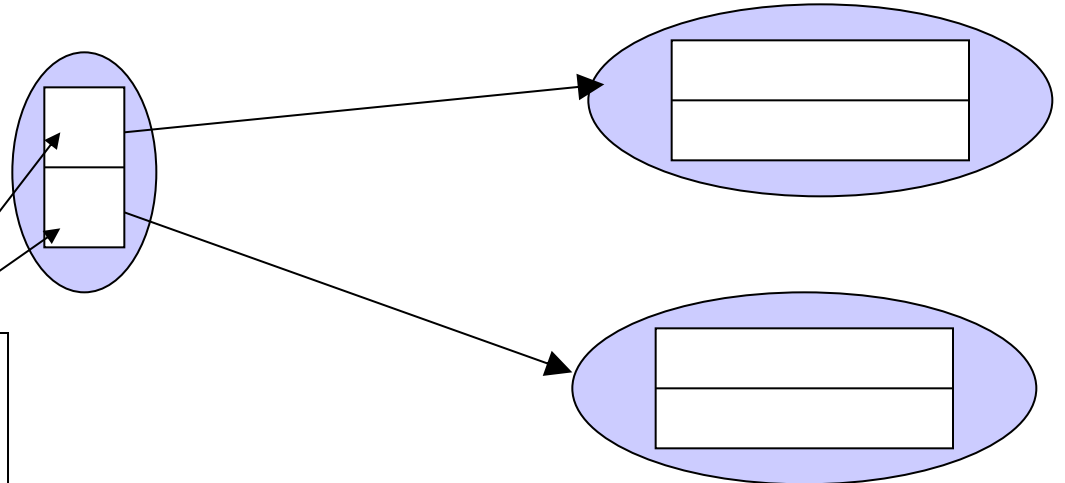
Is:



- In Java

```
Animal[][] arr=  
new Animal[2][2]
```

What is the type of
the object here ?



Static - [1/4]

- **Member data** - Same data is used for all the instances (objects) of some Class.

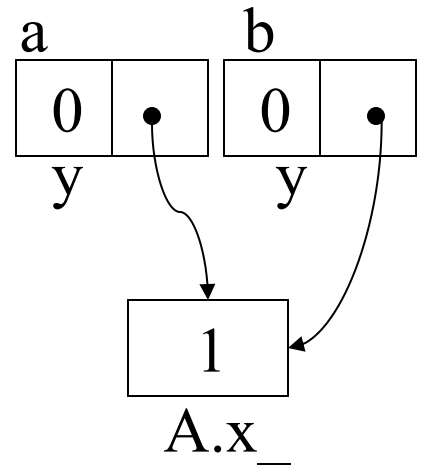
```
Class A {  
    public int y = 0;  
    public static int x_ = 1;  
};
```

*Assignment performed
on the first access to the
Class.
Only one instance of 'x'
exists in memory*

```
A a = new A();  
A b = new A();  
System.out.println(b.x_);  
a.x_ = 5;  
System.out.println(b.x_);  
A.x_ = 10;  
System.out.println(b.x_);
```

Output:

1
5
10



Static - [2/4] cont.

Usage:

```
TeaPot tp1 = new TeaPot(Color.RED);  
  
TeaPot tp2 = new TeaPot(Color.GREEN);  
  
System.out.println("We have " +  
    TeaPot.howManyTeaPots() + "Tea Pots");
```


Static - [3/4]

- **Block**

- Code that is executed in the first reference to the class.
- Several static blocks can exist in the same class
(Execution order is by the appearance order in the class definition).
- Only static members can be accessed.

```
class RandomGenerator {  
    private static int seed_;  
  
    static {  
        int t = System.getTime() % 100;  
        seed_ = System.getTime();  
        while(t-- > 0)  
            seed_ = getNextNumber(seed_);  
    }  
}
```

String is an Object

- Constant strings as in C, does not exist
- The function call `foo("Hello")` creates a String object, containing “Hello”, and passes reference to it to `foo`.
- There is no point in writing :

```
String s = new String("Hello");
```

- The String object is a constant. It can't be changed using a reference to it.

Static - [2/4]

- **Member function**

- Static member function can access only static members
- Static member function can be called without an instance.

```
Class TeaPot {  
    private static int numOfTP = 0;  
    private Color myColor_  
    public TeaPot(Color c) {  
        myColor_ = c;  
        numOfTP++;  
    }  
    public static int howManyTeaPots()  
    { return numOfTP; }  
  
    // error :  
    public static Color getColor()  
    { return myColor_; }  
}
```

Flow control

Basically, it is exactly like c/c++.

if/else

```
If (x==4) {  
    // act1  
} else {  
    // act2  
}
```

do/while

```
int i=5;  
do {  
    // act1  
    i--;  
} while(i!=0);
```

for

```
int j;  
for(int i=0;i<=9;i++)  
{  
    j+=i;  
}
```

switch

```
char  
c=IN.getChar();  
switch(c) {  
    case 'a':  
        // act1  
        break;  
    default:  
        // act2  
}
```

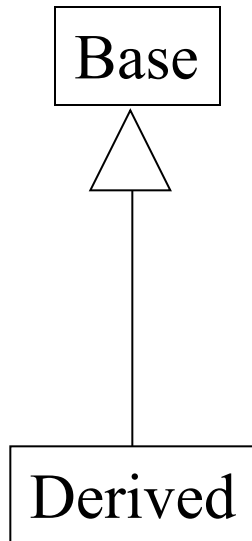
Packages

- Java code has hierarchical structure.
- The environment variable CLASSPATH contains the directory names of the roots.
- Every Object belongs to a package (‘package’ keyword)
- Object full name contains the name full name of the package containing it.

Access Control

- ***public*** member (function/data)
 - Can be called/modified from outside.
- ***protected***
 - Can be called/modified from derived classes
- ***private***
 - Can be called/modified only from the current class
- ***default (if no access modifier stated)***
 - Usually referred to as “Friendly”.
 - Can be called/modified/instantiated from the same package.

Inheritance



```
class Base {
    Base() {}
    Base(int i) {}
    protected void foo() {...}
}

class Derived extends Base {
    Derived() {}
    protected void foo() {...}
    Derived(int i) {
        super(i);
        ...
        super.foo();
    }
}
```

As opposed to C++, it is possible to inherit only from ONE class.

Pros avoids many potential problems and bugs.

Cons might cause code replication

Polymorphism

- Inheritance creates an “is a” relation:

For example, if B inherits from A, then we say that “B is also an A”.

Implications are:

- access rights (Java forbids reducing access rights) - derived class can receive all the messages that the base class can.
- behavior
- precondition and postcondition

Inheritance (2)

- In Java, all methods are virtual :

```
class Base {
    void foo() {
        System.out.println("Base");
    }
}
class Derived extends Base {
    void foo() {
        System.out.println("Derived");
    }
}
public class Test {
    public static void main(String[] args) {
        Base b = new Derived();
        b.foo();    // Derived.foo() will be activated
    }
}
```

Inheritance (3) - Optional

```
class classC extends classB {
    classC(int arg1, int arg2){
        this(arg1);
        System.out.println("In classC(int arg1, int arg2)");
    }
    classC(int arg1){
        super(arg1);
        System.out.println("In classC(int arg1)");
    }
}

class classB extends classA {
    classB(int arg1){
        super(arg1);
        System.out.println("In classB(int arg1)");
    }
    classB(){
        System.out.println("In classB()");
    }
}
```

Inheritance (3) - Optional

```
class classA {
    classA(int arg1){
        System.out.println("In classA(int arg1)");
    }
    classA(){
        System.out.println("In classA()");
    }
}

class classB extends classA {
    classB(int arg1, int arg2){
        this(arg1);
        System.out.println("In classB(int arg1, int arg2)");
    }
    classB(int arg1){
        super(arg1);
        System.out.println("In classB(int arg1)");
    }
}

class B() {
    System.out.println("In classB()");
}
}
```

Method Overloading

- Two methods in a class can share a name, as long as they have different parameter declarations.
- Example:

```
class Overload {  
    void display() {  
        System.out.println("<nothing>");  
    }  
    void display(int x) {  
        System.out.println(x);  
    }  
}
```

Overloading Methods

- In Java it is possible to define two or more methods within the same class that share the same name, **as long as their parameter declarations are different**
- When an overloaded method is invoked, Java uses the **type and/or number of arguments** as its guide to determine which version of the overloaded method to actually call
- The return type alone is **insufficient** to distinguish two versions of a method

```
class Overload {  
    void test() {  
        System.out.println("No parameters");}  
    // Overload test for one integer parameter.  
    void test(int a) {  
        System.out.println("a: " + a);}  
    // Overload test for two integer parameters.  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " " + b);}  
  
    // overload test for a double parameter  
    double test(double a) {  
        System.out.println("double a: " + a);  
        return a*a;  
    }  
}
```

```
class OverloadDemo {  
  
    public static void main(String args[]) {  
        Overload ob = new Overload();  
        double result;           // call all versions of test()  
  
        ob.test();  
        ob.test(10);  
        ob.test(10, 20);  
        result = ob.test(123.25);  
        System.out.println("Result of ob.test(123.25): " +  
                             result);  
    }  
}
```

- In some cases Java's **automatic type conversions** can play a role in overload resolution
- Java will employ its automatic type conversions **only if no exact match is found**

// Automatic type conversions apply to overloading.

```
class Overload2 {  
    void test() {  
        System.out.println("No parameters");  
    }  
    // Overload test for two integer parameters.  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " " + b);  
    }  
    // overload test for a double parameter  
    void test(double a) {  
        System.out.println("Inside test(double) a: " + a);  
    }  
}
```

```
class OverloadTypeCast {  
    public static void main(String args[]) {  
        Overload2 ob = new Overload2();  
        int i = 88;  
        ob.test();  
        ob.test(10, 20);  
        ob.test(i); // this will invoke test(double)  
        ob.test(123.2); //this will invoke test(double)  
    }  
}
```

This program generates the following output:

No parameters

a and b: 10 20

Inside test(double) a: 88

Inside test(double) a: 123.2

Constructors

- Used to initialize an object when it is created
- Syntactically it is like a method.
- Its name is the name of the class.
- Have no return type, not even void
- Automatically called when an object is created.
- Example constructor for **Vehicle** class:

```
Vehicle() {  
    fuelCap = 12;  
    mpg = 25;  
    passengers = 5;  
}
```

Default Constructors

Consider

Vehicle van = new Vehicle();

- **new Vehicle()** is calling the **Vehicle()** constructor
- When you do not explicitly define a constructor for a class, then Java creates a default constructor for the class
- The **default constructor** automatically initializes all instance variables to zero

Constructors with Parameters

- Example:

- In the **Vehicle** class:

```
Vehicle(int p, int f, int m) {  
    passengers = p; fuelCap = f; mpg = m;  
}
```

- In the **VehicleDemo's main()** method:

```
Vehicle car = new Vehicle(5, 12, 25);  
Vehicle van = new Vehicle(7, 24, 21);
```

Constructor Overloading

- You can overload constructors.
- Example:

```
class Overload{  
    int data;  
    Overload(int x) {  
        data = x;  
    }  
    Overload(int x, int y) {  
        data = x+y;  
    }  
}
```

The Keyword **this**

- **this** is an implicit argument that refers to the object on which the method is called.
- **this** is useful for making it clear that you are referring to an instance variable.
- Example in **Vehicle** class:

```
double fuelNeeded(int distance) {  
    return (double) distance / this.mpg;  
}
```

Instance variable hiding

- It is illegal in java to declare two local variables with the same name inside the same or enclosing scopes.
- Interestingly, you can have local variables, including formal parameters to methods, **which overlap with the names of the class instance variables**. We can see the following example for this explanation.

Example:

```
Vehicle(int p, int f, int m) {  
    passengers = p; fuelCap = f; mpg = m;  
}
```


Instance variable hiding

- However, when a local variable has the same name as an instance variable, the **local variable hides the instance variable**. This is why `passengers`, `fuelCap`, `mpg` were not used as the names for the parameters to the `Vehicle()` constructor inside the `Vehicle` class. If they had been, then `passengers` would have referred to the formal parameter, hiding the instance variable `passengers`.

Example:

```
Vehicle(int passengers, int f, int m) {  
    passengers = passengers; // passengers refers to formal parameter  
    fuelCap = f;  
    mpg = m;  
}
```

Instance variable hiding

- While it is usually easier to simply use different names, there is another way around this situation. Because this lets you refer directly to the object, you can use it to resolve any name space collisions that might occur between instance variables and local variables.

// Use **this** to resolve **name-space collisions**.

```
Vehicle(double passengers, double fuelCap, double mpg) {  
    this. passengers = passengers;  
    this. fuelCap= fuelCap;  
    this.mpg = mpg;  
}
```

Abstract

- ***abstract*** member function, means that the function does not have an implementation.
- ***abstract*** class, is class that can not be instantiated.

```
AbstractTest.java:6: class AbstractTest is an abstract class.  
It can't be instantiated.
```

```
    new AbstractTest();  
    ^
```

```
1 error
```

NOTE:

An abstract class is **not** required to have an abstract method in it. But any class that has an abstract method in it or that does not provide an implementation for any abstract methods declared in its superclasses **must** be declared as an abstract class.

Abstract - Example

```
package java.lang;
public abstract class Shape {
    public abstract void draw();
    public void move(int x, int y) {
        setColor(BackgroundColor);
        draw();
        setCenter(x,y);
        setColor(ForegroundColor);
        draw();
    }
}
```

```
package java.lang;
public class Circle extends Shape {
    public void draw() {
        // draw the circle ...
    }
}
```

Interface

Interfaces are useful for the following:

- Capturing similarities among unrelated classes without artificially forcing a class relationship.
- Declaring methods that one or more classes are expected to implement.
- Revealing an object's programming interface without revealing its class.

Interface

- abstract “class”
- Helps defining a “usage contract” between classes
- All methods are public
- Java’s compensation for removing the multiple inheritance. You can “inherit” as many interfaces as you want.

* - The correct term is “to implement”
an interface

Interface

```
interface IChef {  
    void cook(Food food);  
}
```

```
interface BabyKicker {  
    void kickTheBaby(Baby);  
}
```

```
interface SouthParkCharacter {  
    void curse();  
}
```

```
class Chef implements IChef, SouthParkCharacter {  
    // overridden methods MUST be public  
    // can you tell why ?  
    public void curse() { ... }  
    public void cook(Food f) { ... }  
}
```

* access rights (Java forbids reducing of access rights)

When to use an interface ?

Perfect tool for encapsulating the classes inner structure. Only the interface will be exposed